

Week 7 — Control Flow, Exceptions, and Privilege

Target time: approximately 3 hours

Rhythm: 75 minutes concept and tracing · 75 minutes lab · 30 minutes diagram, explanation, and review

The CPU is executing your loop. A timer device asks for attention. At nearly the same moment, your process executes a system call.

Who is allowed to change what runs next—and how can the machine enter the kernel without trusting the program to choose an arbitrary kernel address?

That is this week's problem. A useful computer must do more than calculate the next sequential result. It must choose, repeat, call, react, and cross a protection boundary under controlled conditions.

The path is:

```
compare values
  → record condition state
  → choose the next PC
  → build loops and calls
  → interrupt ordinary control flow
  → enter a trusted handler through a hardware-defined gate
  → restore the interrupted context
```

By the end, you should be able to explain how the CPU stops normal execution and safely runs an operating-system handler. “The kernel takes over” is not enough. You should be able to name what causes the transfer, how the target is chosen, what state must be saved, why privilege changes, and how execution returns.

1. The historical problem: useful machines cannot only march forward

An early automatic calculator could perform a predetermined sequence of operations. That is already valuable, but consider long division: subtract a divisor repeatedly **while** the remainder is large enough. Or consider a payroll calculation: choose one formula **if** a condition holds and another otherwise.

The missing capability is not another arithmetic operation. It is the power to choose the next operation from a result already computed.

A stored-program CPU normally keeps the address of the next instruction in a **program counter**—RIP in x86-64 terminology. If every instruction merely advanced that address, the program would be a straight line:

```
PC = 0x1000  execute
PC = 0x1004  execute
PC = 0x1008  execute
...
```

Control flow appears when an instruction can replace the ordinary next PC:

```
ordinary instruction: PC ← address after this instruction
branch taken:       PC ← branch target
call:               save return location; PC ← function entry
return:             PC ← saved return location
exception/interrupt: save interrupted state; PC ← trusted handler
```

These mechanisms differ in purpose and authority, but all answer the same physical question: **which instruction address will be presented for fetch next?**

2. Compare, flags, and conditional branches

Suppose C says:

```
if (x < limit)
    ++x;
```

At the machine level there is usually no little object named `if`. The compiler emits operations that establish a condition and then conditionally change control flow.

On x86-64, `cmp source, destination` conceptually computes:

```
destination - source
```

It discards the arithmetic result but updates selected bits in `RFLAGS`. Important examples are:

- `ZF`, zero flag: the conceptual result was zero;
- `SF`, sign flag: the high result bit was one;
- `CF`, carry flag: unsigned subtraction required a borrow;
- `OF`, overflow flag: signed arithmetic overflowed.

A following `jcc` instruction tests a condition derived from those flags. For example, in AT&T syntax:

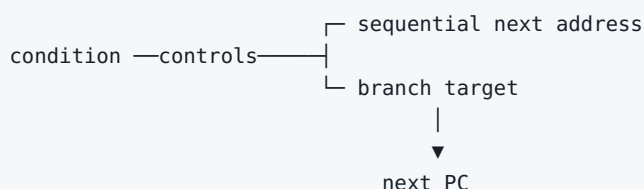
```
cmp %esi, %edi    # conceptual calculation: edi - esi
jl  smaller_signed # signed less-than: SF differs from OF
jb  smaller_unsigned # unsigned below: CF is set
je  equal         # ZF is set
```

Signed and unsigned comparisons use the **same bits** but ask different questions. That is why `jl` and `jb` are different. `SF` alone cannot reliably mean signed less-than because signed overflow can invert the apparent sign; the signed condition accounts for `OF`.

Some ISAs expose condition-code flags; some use compare-and-branch instructions or write comparison results into general registers. Do not turn the x86 arrangement into a universal law. The universal idea is that the machine derives a Boolean condition and uses it to select the next PC.

A branch is a PC multiplexer in architectural form

At a simple conceptual level:



Real processors predict branches and may fetch or execute speculatively. Those mechanisms improve performance; they do not change the architectural promise. When the instruction retires, the visible state must match the path the ISA says was taken.

3. A loop is a backward conditional transfer

Consider:

```
int sum_to(int n)
{
    int sum = 0;
    for (int i = 0; i < n; ++i)
        sum += i;
    return sum;
}
```

One possible unoptimized shape is:

```
    sum = 0
    i = 0
test:
    compare i with n
    branch to done if i >= n
body:
    sum = sum + i
    i = i + 1
    jump back to test
done:
    return sum
```

Nothing in hardware recognizes the source-language phrase “for loop.” The loop exists as:

1. state holding `i`, `sum`, and `n`;
2. arithmetic that updates that state;
3. a comparison;
4. a conditional branch to leave;
5. a backward control transfer to repeat.

Trace three iterations by hand

For `n = 3`:

test	i	sum before body	i < n	next PC choice
1	0	0	true	body
2	1	0	true	body
3	2	1	true	body
4	3	3	false	done

The source loop feels like one repeating object. The CPU sees a sequence of individual instructions and repeatedly selects an earlier instruction address.

Calls and returns: a preview

A direct `call` also changes `RIP`, but first preserves a return address. On x86-64 a near `call` pushes the address after the call onto the current stack, then transfers to the callee. `ret` obtains the saved address and places it back into `RIP`.

```
caller: call f
        |
        |— save address of "after call"
        |— RIP ← entry of f

f:      ... work ...
        ret —————→ RIP ← saved address
```

This is a preview, not yet a complete ABI lesson. Arguments, preserved registers, stack alignment, frames, and security hardening come later. For this week, notice the contrast:

- branches choose among program-specified targets;
- calls preserve an ordinary program return point;
- exception entry uses targets and state rules configured for protected handling, not a return address supplied freely by untrusted user code.

4. Why privilege exists

Imagine every process could:

- program the disk controller;
- disable interrupts forever;
- rewrite another process's page tables;
- install its own exception target;
- read all physical memory.

One bad pointer or hostile program could destroy every protection the operating system claims to provide. Multiprogramming therefore needs more than a convention saying “applications should behave.” The processor must enforce authority.

Modern architectures provide execution modes. In ordinary x86-64 Linux:

```
ring 3 / CPL 3   user programs: restricted
ring 0 / CPL 0   kernel: privileged
```

x86 defines four rings, numbered 0 through 3, but mainstream Linux primarily uses ring 0 and ring 3. “Ring 1 and ring 2 must contain drivers” is not a rule of x86-64 Linux. Other architectures use different language and structures: AArch64 commonly runs applications at EL0 and the kernel at EL1, with EL2 available for a hypervisor.

Privilege is CPU state, not a moral description of source code. While executing with insufficient privilege, certain instructions or accesses are blocked by hardware. Page-table permissions also distinguish user-accessible from supervisor-only mappings.

User versus kernel is more than two address ranges

Keep three related ideas separate:

1. **current privilege:** which operations the CPU permits now;
2. **address translation and permissions:** which memory mappings this execution may access;
3. **software policy:** what the kernel chooses to allow after validating a request.

Changing privilege does not automatically grant an application permanent kernel authority. On a system call, the CPU begins executing a kernel entry point under kernel control. The kernel validates arguments and performs a specific service, then returns to restricted execution.

5. Four ways normal sequencing can change

The word “exception” is overloaded across architecture manuals, operating systems, programming languages, and conversation. Start with timing.

Synchronous exception

A synchronous event is caused by, and associated with, the instruction being executed. Repeat the same architectural state and instruction, and the event is tied to that execution.

Examples:

- invalid opcode;
- divide error;
- page fault;
- general-protection fault;
- a deliberately executed system-call instruction.

Asynchronous interrupt

An asynchronous interrupt originates outside the current instruction stream. A timer expires, a network device completes work, or an interrupt controller delivers a pending request. The CPU normally recognizes it at an architecturally defined boundary between instructions.

```
exception: "this instruction encountered a condition"
interrupt: "an external agent needs attention"
```

The interrupted instruction did not cause the network packet to arrive. That distinction matters even though both mechanisms eventually vector into privileged software.

x86's trap, fault, and abort classification

The Intel architecture classifies **exceptions** by where the saved instruction pointer points and whether restart is meaningful:

- **fault:** reported before the instruction completes; saved state generally allows the handler to correct the cause and restart the instruction;
- **trap:** reported after the instruction completes; saved instruction pointer generally names the next instruction;
- **abort:** severe condition for which the precise instruction location may not be reliably reported and restart is not generally possible.

A page fault is the classic fault: Linux may install a missing mapping and retry the instruction. A debug single-step event is a classic trap. Machine check conditions can be abort-class events.

Be careful with the word **trap**. Some textbooks use “trap” broadly for any synchronous transfer, some operating systems call their saved frame a trap frame, and architecture manuals classify specific events differently. ARM documentation uses its own exception terminology. State which vocabulary you mean instead of assuming one universal taxonomy.

Also do not confuse a software `raise(SIGILL)` with a hardware invalid-opcode exception. `raise` asks the operating system to deliver a Unix signal; it does not execute an invalid instruction.

6. Vectoring: a controlled answer to “where next?”

If user code could respond to an exception by naming any ring-0 address and stack, protection would disappear. The hardware and kernel therefore prearrange legal entry paths.

On x86, exceptions and interrupts have vector numbers. The kernel installs descriptor information in an Interrupt Descriptor Table (IDT). Conceptually:



Exact saved fields depend on the event and transition. On x86, hardware entry preserves enough control state for the architectural return; entry assembly saves additional general registers into a kernel-defined frame. Some exceptions push an error code and others do not. Do not simplify this to “the CPU saves every register.”

Why a kernel stack?

The current user stack belongs to untrusted process state. It may be full, unmapped, maliciously positioned, or writable by user code. On entry from user mode, the kernel needs a trusted stack on which to preserve context and run nested C/assembly call sequences. Linux associates kernel execution state with the current task and enters on kernel-controlled stack storage.

The kernel stack is not “where the entire process is stored.” It is working space for privileged execution, including saved register/context data and kernel call frames.

Return from exception

After handling, architecture-specific return machinery restores control state and lowers privilege as appropriate. On x86 this family includes `iret / iretq`; fast syscall paths commonly use `sysret` when conditions allow. The kernel must validate and arrange the return state—it does not blindly trust arbitrary values merely because they came from user memory.

7. Concrete cases

Invalid opcode: `#UD` can lead to `SIGILL`

Suppose user code attempts bytes that do not encode a valid instruction for the current mode.

```
decoder rejects instruction
  → x86 invalid-opcode exception #UD (vector 6)
  → CPU enters kernel exception path
  → Linux decides the user task cannot continue
  → Linux delivers SIGILL
  → signal disposition handles it or terminates the process
```

`#UD` is a CPU exception. `SIGILL` is a Unix process signal chosen by the OS as the user-visible consequence. They are not the same object and the mapping is OS policy, not an intrinsic wire from the CPU to a C signal handler.

The accompanying lab uses `raise(SIGILL)` by default. That safely exercises signal delivery and handler constraints without embedding raw invalid opcode bytes. Therefore it begins at the signal-delivery part of this diagram; it does **not** claim to generate `#UD`.

Divide error: `#DE` and a possible `SIGFPE`

On x86, integer divide by zero or an unrepresentable quotient can produce the divide-error exception `#DE` (vector 0). Linux can translate a user-mode case into `SIGFPE`.

Do not write `1 / zero` in portable C and claim it is a reliable CPU experiment. Integer division by zero is undefined behavior in C: the compiler is not required to emit a hardware divide. The lab therefore uses `raise(SIGFPE)` as a safe signal-flow demonstration and labels it honestly. A hardware `#DE` experiment belongs in controlled architecture-specific assembly with its assumptions stated.

Protection example: `#GP`

A user-mode attempt to execute a privileged operation can cause a general-protection exception, `#GP` on x86. The hardware refuses the operation before letting ring-3 code alter protected machine state. Linux then decides how the user task should observe that failure; the resulting signal depends on the particular condition and kernel path.

Page fault: often normal

A page fault `#PF` does not mean “segmentation fault.” It means address translation or permission checking needs privileged attention.

```
#PF
├─ valid VMA, lazily allocated page
│   kernel maps a page → retry instruction → program notices nothing
├─ copy-on-write mapping
│   kernel creates private writable page → retry
└─ no valid mapping or forbidden access
    kernel cannot repair → often deliver SIGSEGV
```

Demand paging depends on a fault being restartable. The same architectural mechanism can be routine or fatal; the kernel's memory policy and process mapping decide which.

Hardware interrupt flow

Take a network receive interrupt:

1. NIC receives data and records device state.
2. NIC/MSI path notifies the interrupt controller/CPU.
3. CPU recognizes a pending, unmasked interrupt at an allowed boundary.
4. Vectoring saves return state and enters a kernel interrupt entry point.
5. Entry code saves additional context on a kernel stack.
6. Driver/kernel handler acknowledges and services or schedules work.
7. Return-from-interrupt restores the interrupted context.
8. The CPU resumes the selected task—or scheduling may choose another task.

The handler did not arrive by calling a normal C function from user code. Hardware established a protected control transfer. Likewise, interrupt return is not an ordinary `ret`: it restores privilege and machine state associated with the interrupted context.

8. System-call privilege transition

A system call is a deliberate request, but it still needs a controlled boundary crossing.

For a typical x86-64 Linux system call:

```
user code / libc wrapper
  | place syscall number and arguments per ABI
  ▼
SYSCALL instruction
  | CPU loads configured kernel entry state and changes privilege
  ▼
architecture entry assembly
  | establish safe kernel context; save registers
  ▼
Linux syscall dispatch
  | validate number, pointers, lengths, permissions
  ▼
requested kernel service
  | result or -errno
  ▼
kernel exit path
  | restore user-visible state
  ▼
SYSRET or another safe return path
  ▼
user mode resumes
```

This is not a conventional function call:

- a normal call stays at the same privilege and uses a program-chosen target;
- a syscall uses CPU entry configuration controlled by the kernel;
- user pointers remain untrusted inputs after entry;
- the kernel may block, schedule, deliver a signal, or return an error before the original thread resumes.

The x86-64 `SYSCALL` mechanism is also not the same entry mechanism as every IDT exception, even though both produce exceptional control flow and a privilege transition. Keep the common concept—controlled entry—separate from the exact architecture-specific path.

9. Observe: compile and trace a loop at instruction level

Use the provided lab:

```
cd /home/jd/Desktop/masterclass

gcc -std=c17 -Wall -Wextra -Wpedantic -Werror -O0 -g \
    weekly/challenges/week-007-exception-flow-lab.c \
    -o /tmp/exception-flow-lab

/tmp/exception-flow-lab normal
```

`-O0` keeps the first trace easy to relate to source. It does not promise one fixed assembly layout; compiler version, ABI, and options matter.

Disassemble before debugging

```
objdump -d -Intel /tmp/exception-flow-lab | less
```

Search inside `less` for `<run_normal_loop>`. Identify:

1. initialization of `sum` and `i`;
2. the compare instruction;
3. the conditional branch controlling exit/repetition;
4. the instruction address targeted by the backward edge.

Then compare optimization:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Werror -O2 -g \
    weekly/challenges/week-007-exception-flow-lab.c \
    -o /tmp/exception-flow-lab-O2
objdump -d -Intel /tmp/exception-flow-lab-O2 | less
```

Do not ask only “where did my `for` statement go?” Ask which observable behavior the compiler had to preserve and which source-level structure it was free to rearrange.

Single-step instructions in GDB

```
gdb -q /tmp/exception-flow-lab
```

At the GDB prompt:

```
set disassembly-flavor intel
break run_normal_loop
run normal
display/i $pc
display/x $eflags
info registers rip eflags
disassemble /m run_normal_loop
si
si
```

Continue with `si` through one compare and branch. Immediately before and after the compare, run:

```
p/x $eflags
info registers
x/6i $pc
```

Record:

- RIP before the compare;
- the relevant values being compared;
- flags after the compare;
- RIP after the branch;
- whether the target or fall-through address became the next PC.

Use `ni` if you want to step over a call such as `printf` rather than entering libc instruction by instruction.

10. Build and observe the exception-flow lab

The actions are selectable:

```
/tmp/exception-flow-lab normal
/tmp/exception-flow-lab ill
/tmp/exception-flow-lab segv
/tmp/exception-flow-lab fpe
/tmp/exception-flow-lab all
```

The signal actions run in forked children. Their handlers use `write(2)` and `_exit(2)`, both suitable for this signal-handler path; they do not call `printf`, `malloc`, or `exit`. The parent waits, reports status, and continues, so the experiment does not crash the shell or the parent lab process.

The `segv` and `fpe` choices deliberately call `raise`. They demonstrate Unix signal flow safely, not real page-fault or divide-error generation. This distinction is part of the exercise.

Trace process and signal behavior

```
strace -f -e trace=process,signal /tmp/exception-flow-lab all
```

Look for:

- `clone` / `fork`-related process creation;
- `rt_sigaction` installing dispositions;
- a signal-send operation used by `raise` (often `tgkill` on Linux);
- `SIGILL`, `SIGSEGV`, and `SIGFPE` delivery;
- child exit and parent `wait4`/wait behavior.

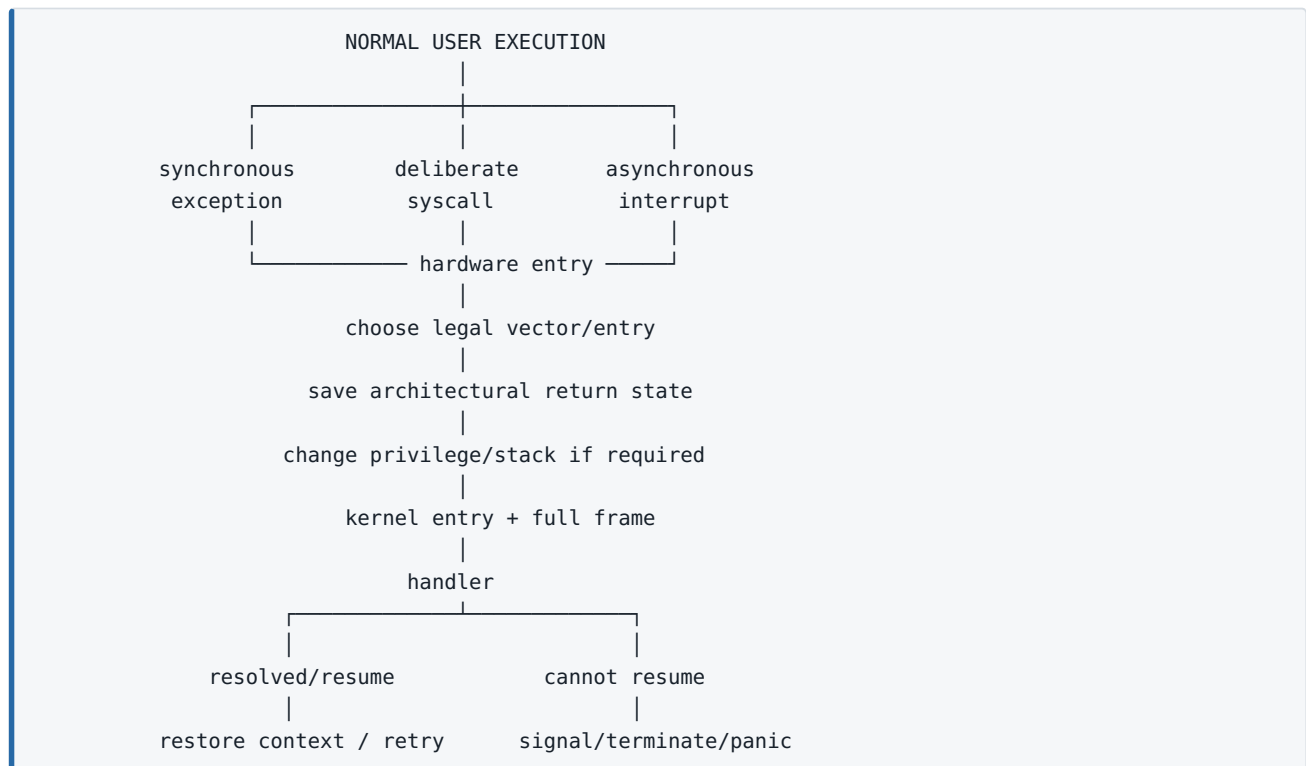
Then observe a real syscall transition at the user-visible boundary:

```
strace -e trace=write /tmp/exception-flow-lab normal
```

`strace` shows syscall entry/return from the tracer's viewpoint. It does not directly print every hidden CPU entry instruction, privilege bit change, or kernel-stack operation. Use the documented transition model to explain what must occur between the userspace call site and the returned syscall result.

11. Draw: exception-flow diagram

Redraw this without copying labels, then annotate one `#UD` path and one hardware-interrupt path:



Add these precision notes:

- asynchronous interrupts are not caused by the current instruction;
- not every exception changes privilege if it occurs while already in the kernel;
- not every page fault is fatal;
- a Unix signal is a software abstraction, not a CPU exception vector;
- the exact saved frame and return instruction are architecture-specific.

12. Mastery: Explain · Draw · Observe · Build · Why

Explain

Without notes, answer:

How can the CPU stop normal execution and safely run an operating-system handler?

A complete explanation should connect:

1. event recognition;
2. vector or controlled entry selection;
3. protected privilege transition;
4. trusted kernel stack and saved context;
5. kernel entry code and handler;
6. handler decision: repair, resume, signal, schedule, or terminate;
7. validated return-from-exception restoring the prior context.

Draw

Draw both:

- compare → flags/condition → next-PC choice → loop;
- user execution → event → vector → saved context/kernel stack → handler → return.

Observe

Collect:

- one GDB trace showing the PC taking or not taking a loop branch;
- one `objdump` excerpt identifying the backward edge;
- one `strace -f` observation showing child signal delivery and parent survival.

Build

Extend only after understanding the supplied program:

1. add a `usr1` action for `SIGUSR1`;
2. keep the handler async-signal-safe;
3. run it in a child;
4. predict the trace before running;
5. explain why this is still a Unix-signal experiment, not a CPU exception.

Why?

Write short answers:

1. Why can a conditional branch be described as choosing a new PC?
 2. Why do signed and unsigned branches inspect conditions differently?
 3. Why is a user stack not sufficient as the only stack for kernel entry?
 4. Why can a page fault be a successful part of normal execution?
 5. Why is `#UD → Linux → SIGILL` not the same as `raise(SIGILL)`?
 6. Why must the return path restore privilege as well as an instruction address?
-

Mental model

```
sequential execution
+ conditional PC selection
  = decisions and loops

ordinary calls
+ saved program return address
  = reusable procedures

hardware event recognition
+ protected vectoring
+ trusted saved context and kernel stack
+ privileged handler
+ architectural return
  = safe exceptional control flow
```

The sentence to retain is:

The CPU does not let an event merely “jump into the kernel.” It recognizes a defined event, selects a kernel-configured entry path, preserves return state, enforces privilege and stack rules, runs trusted entry/handler code, and returns only through an architecture-defined restoration path.

References used selectively

Primary architecture and Linux references:

- Intel, *Intel® 64 and IA-32 Architectures Software Developer's Manual*, Volume 3A, Chapter 6 (“Interrupt and Exception Handling”) and Chapter 7 (“Task Management”), especially the exception classes, IDT, privilege transitions, stacks, and IRET; Volume 2 for CMP, conditional jumps, SYSCALL, SYSRET, and IRET instruction semantics: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
- Linux kernel documentation, “Kernel Entries,” including x86 syscall and interrupt/exception entry categories: <https://docs.kernel.org/entry/entry.html>
- Linux x86-64 syscall entry implementation, arch/x86/entry/entry_64.S: https://github.com/torvalds/linux/blob/master/arch/x86/entry/entry_64.S
- Linux man-pages: sigaction(2), signal(7), signal-safety(7), raise(3), fork(2), and waitpid(2): <https://man7.org/linux/man-pages/>
- strace(1) manual: <https://man7.org/linux/man-pages/man1/strace.1.html>

Local reading:

- Kaiwan N. Billimoria, *The Linux Operating System: A Brief on Its Architecture*, “Execution Privilege Levels,” “Arch-specific issuing of system calls,” and “Flow of a Process,” local PDF pages 16–22: <source-materials/library/training/kaiwantech/kernel-internals-2024/01-linux-system-architecture.pdf>
- Randal E. Bryant and David R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., Chapter 3 on machine-level control and Chapter 8, §§8.1–8.5 on exceptional control flow, exceptions, processes, and signals: <source-materials/library/books/01-foundations->

and-architecture/computer-systems-programmers-perspective-bryant-ohallaron-3e-global.pdf

- Yale N. Patt and Sanjay J. Patel, *Introduction to Computing Systems*, 2nd ed., Chapter 4 on control instructions and Chapters 8–9 on I/O, interrupts, privilege, and operating-system service: [source-materials/library/books/01-foundations-and-architecture/introduction-to-computing-systems-patt-patel-2e.pdf](#)

Next bridge: x86-64 assembly foundations will make registers, addressing, instruction encodings, and debugger traces more precise; stack frames and calling conventions will then complete the ordinary `call/ret` story.